"From Concepts to Real Projects!"

@codedsoul_05

Learn Build Apply Grow!

Complete

LangChain Cheat Sheet + 100 Interview Questions

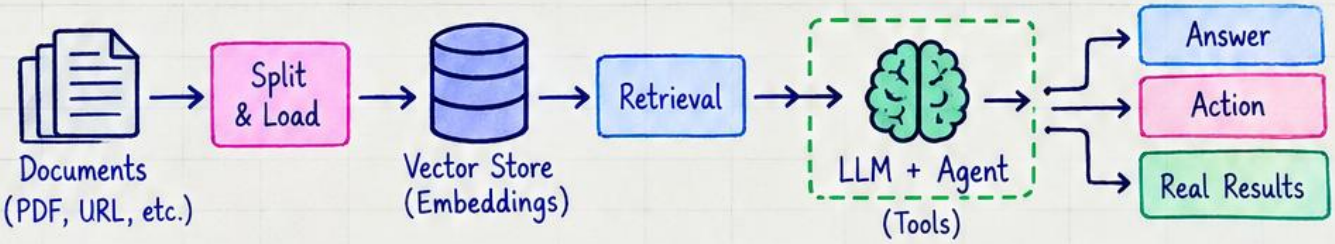
Everything you need to Build, Learn and Crack Interviews!



LangChain

- ☑ Core Concepts
- ☑ Components & APIs
- ☑ RAG + Agents
- ☑ Tools & Integrations
- ☑ Production Best Practices
- ☑ 100 Interview Questions
- ☑ Real World Project Insights

"Turn LLMs into Real World Applications"



Understand Key Concepts

Build Real Projects

Prepare for Interviews

Apply in Real World

"Smaller Building Blocks. Bigger Possibilities."

@codedsoul_05

1. LangChain Introduction + Why LangChain ?

@codedsoul_05

1. What is LangChain ?

- LangChain is an open source framework to build applications with Large Language Models (LLMs).
- It provides tools, abstractions and prebuilt components to connect LLMs with real world data, tools and workflows.
- It helps developers go from experiments to production faster.
- Supports multiple LLMs (OpenAI, Anthropic, Google, Meta, etc.) and integrates with many tools and data sources.

> "LangChain makes it easier to build LLM powered applications."

2. Why LangChain ?

- Simplifies LLM application development
- Provides prebuilt components (chains, agents, tools, memory, etc.)
- Easy integration with external data (PDFs, databases, APIs, etc.)
- Works with multiple LLM providers
- Helps in building RAG (Retrieval Augmented Generation) applications
- Supports complex workflows and agent systems
- Actively maintained with a large community
- Used by startups and enterprises in production

> "Focus on building your idea, not on reinventing the plumbing."

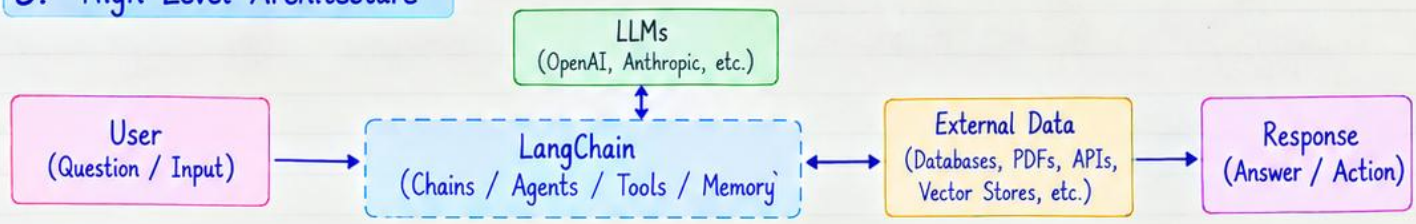
3. Key Features of LangChain

- Modular and flexible architecture
- Chains for building LLM workflows
- Agents to use tools and perform actions
- Memory to maintain conversation context
- Integrations with vector stores, databases, APIs and more
- LangGraph for building stateful multi agent workflows
- LangSmith for debugging, tracing and evaluation
- Production ready with extensive documentation

4. LangChain Ecosystem

LangChain	Core framework for building LLM applications
LangGraph	Build stateful and custom agent workflows
LangSmith	Tracing, debugging, evaluation and monitoring
LangServe	Deploy LangChain applications as APIs
LangChain Hub	Prebuilt prompts, chains and agents
Integrations	Connect with OpenAI, Anthropic, Google, Vector DBs, APIs, tools and more

5. High Level Architecture



6. Real World Use Cases

- Chat with your own documents (PDFs, notes, reports)
- Build AI powered customer support bots
- Connect LLMs with databases to answer business questions
- Automate tasks using agents and tools
- Build research assistants
- Create domain specific copilots (education, healthcare, finance, etc.)

LangChain is not just a library, it is a toolkit to build real world LLM applications.

Build Learn Create With LangChain !

Same Concepts Bigger Opportunities !

Set up
Build
Experiment
Learn !

2. Installation + Project Setup

@codedsoul_05

Small
Setup
Big Possibilities !

1. Prerequisites

- Basic knowledge of Python
- Familiarity with terminal / command line
- Python 3.9 or above (recommended 3.10+)
- pip (comes with Python)
- A code editor (VS Code recommended)
- Optional: Git for version control

"A good setup saves hours of debugging later."

2. Install Python (Check Version)

Check Python version

```
python --version      # For Windows
python3 --version     # For Mac/Linux
```

You should see Python 3.9 or higher.
If not, download from python.org

3. Create a Virtual Environment

Using venv (recommended)

```
python -m venv .venv      # create venv
```

Activate the environment

```
# Windows
.venv\Scripts\activate
# Mac / Linux
source .venv/bin/activate
```

Your terminal prompt will show (.venv)
when the environment is activated.

4. Install LangChain and Dependencies

Install latest LangChain

```
pip install langchain
```

Common additional packages

```
pip install langchain-openai      # For OpenAI
pip install langchain-community   # Community integrations
pip install langchain-core        # Core components
pip install python-dotenv         # For environment variables
pip install chromadb              # Vector database (optional)
pip install faiss-cpu             # Another vector DB (optional)
```

Install only the packages you need based on your project.

5. Verify Installation

Check installed packages

```
pip list | findstr langchain  # Windows
pip list | grep langchain     # Mac/Linux
```

Test in Python

```
python
>>> import langchain
>>> print(langchain.__version__)
>>> exit()
```

If no error, LangChain is installed
successfully.

6. Project Folder Structure

```
my-langchain-app/
├── .venv/          # virtual environment
├── .env            # environment variables
├── app/            # source code
├── notebooks/      # experiments (optional)
├── data/           # data files (optional)
├── requirements.txt # dependencies
└── README.md       # project description
```

7. Create a .env File (For API Keys)

Store sensitive keys in a .env file (do not hardcode).

.env

```
OPENAI_API_KEY=your_openai_key_here
LANGCHAIN_TRACING_V2=true      # optional
LANGCHAIN_API_KEY=your_langsmith_key  # optional
```

8. Test a Simple LangChain Setup

Create a file app/test.py and add the following code:

```
from dotenv import load_dotenv
load_dotenv()

from langchain.chat_models import ChatOpenAI
from langchain.schema import HumanMessage

llm = ChatOpenAI(model="gpt-3.5-turbo")
response = llm.invoke([HumanMessage(content="Hello from LangChain!")])
print(response.content)
```

→ Run the file

```
python app/test.py
```

You should see a reply
from the model !

"Setup is the first step towards turning ideas into real world applications."

Good
Tools
Better
Projects !

Same
Concepts
Bigger
Impact !

Same Concepts Bigger Opportunities !

3. Models + LLMs + Chat Models

@codedsoul_05

Learn Build Apply Grow !

1. What are Models in LangChain ?

- Models are the core of LangChain.
- They act as an interface between your application and different LLM providers (OpenAI, Anthropic, Google, etc.).
- LangChain provides a unified way to work with different models using the same API.
- You can easily switch between models without changing your application logic.

"Same code, different model.
That's the power of abstraction."

2. Types of Models

LLMs	General purpose text completion models (e.g. GPT, Claude, Llama)
Chat Models	Optimized for chat conversations (e.g. ChatOpenAI, ChatAnthropic)
Embedding Models	Convert text to vectors (e.g. OpenAIEmbeddings)
Multimodal Models	Can handle text, images, audio (e.g. GPT-4V, Gemini)
Local Models	Run on your own machine (e.g. Ollama, Llama.cpp)

3. Popular LLM Providers

Provider	Model Name (Example)	Use Case
OpenAI	gpt-3.5-turbo gpt-4o	General purpose, chat, tools
Anthropic	claude-3-haiku claude-3-sonnet	Safer, long context, reasoning
Google	gemini-pro gemini-1.5-flash	Multimodal, large context
Meta	llama-3	Open source, local / custom
Mistral	mistral-large mistral-medium	High performance, open models
Cohere	command-r command-r-plus	Enterprise use cases
Ollama	llama3, mistral, etc.	Run locally

4. Using LLM (Text Completion)

```
from langchain.llms import OpenAI

llm = OpenAI(
    model="gpt-3.5-turbo-instruct",
    temperature=0.7
)

response = llm.invoke("Explain MLOps in simple words.")
print(response)
```

LLM is used for single turn text completion (not chat based).

5. Using Chat Model (Recommended)

```
from langchain.chat_models import ChatOpenAI
from langchain.schema import HumanMessage,
 SystemMessage

chat = ChatOpenAI(
    model="gpt-3.5-turbo",
    temperature=0.7
)

messages = [
    SystemMessage(content="You are a helpful assistant."),
    HumanMessage(content="Explain LangChain in simple words.")
]

response = chat.invoke(messages)
print(response.content)
```

Chat Models are most commonly used in modern applications.

6. Passing Parameters

```
chat = ChatOpenAI(
    model="gpt-4o",
    temperature=0.3,        # creativity
    max_tokens=500,         # response length
    top_p=1.0,              # nucleus sampling
    frequency_penalty=0.0,
    presence_penalty=0.0
)
```

Parameter	Meaning
model	Model name to use
temperature	Controls randomenss (0 to 1)
max_tokens	Maximum length of response
top_p	Nucleus sampling
frequency_penalty	Reduces repetitive text
presence_penalty	Encourages new topics

7. Switching Between Models

```
# OpenAI
chat = ChatOpenAI(model="gpt-3.5-turbo")

# Anthropic
from langchain.chat_models import ChatAnthropic
chat = ChatAnthropic(model="claude-3-sonnet-20240229")

# Google (Gemini)
from langchain_google_genai import ChatGoogleGenerativeAI
chat = ChatGoogleGenerativeAI(model="gemini-pro")

# Local (Ollama)
from langchain_community.chat_models import ChatOllama
chat = ChatOllama(model="llama3")
```

8. Model with Streaming (Real Time)

```
chat = ChatOpenAI(model="gpt-3.5-turbo",
                  streaming=True)

for chunk in chat.stream([
    HumanMessage(content="Tell me a story about AI.")
]):
    print(chunk.content, end="", flush=True)
```

Streaming gives real time response (token by token).

9. Best Practices

- Choose the right model for your use case
- Set appropriate temperature
- Use system messages for better control
- Handle errors and retries
- Use streaming for better user experience
- Keep an eye on token limits and cost
- Use environment variables for API keys
- Abstract model selection for easy switching

10. Key Takeaways

- LangChain supports multiple LLM providers
- Chat models are preferred for most applications
- You can easily switch models with minimal code changes
- Tune parameters based on your use case
- Use streaming for real time applications

"Models are just the beginning.
The real power comes from how you use them."

Build Real Projects Gain Real Skills !

From Learning to Real Impact !

Better Prompts Better Results !

4. Prompts + Prompt Templates

@codedsoul_05

Same Concepts Bigger Opportunities !

1. What are Prompts ?

- A prompt is the input text given to an LLM to get a response.
- It can be a simple question or a detailed instruction.
- The quality of the prompt directly affects the quality of the output.
- Good prompts are clear, specific and provide enough context.

"A good prompt turns a general model into a useful assistant."

2. Types of Prompts

Type	Description
Zero Shot	Direct instruction without examples
Few Shot	Provide few examples in the prompt
Chain of Thought	Ask the model to think step by step
Role Based	Assign a specific role (e.g. You are a teacher)
Instruction Based	Give clear task instructions
Context Based	Provide background context
Multimodal	Use text + images (for multimodal models)

"Different tasks need different prompting strategies."

3. Prompt vs Prompt Template

Prompt

A fixed input text given to the model.

```
# Simple Prompt
prompt = "Explain what is MLOps
in simple words."
```

Prompt Template

A reusable prompt with placeholders for dynamic values.

```
from langchain.prompts import PromptTemplate
template = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic} in simple words."
)
prompt = template.format(topic="MLOps")
print(prompt)
```

"Prompt templates help you reuse and parameterize your prompts."

4. Creating a Prompt Template

```
from langchain.prompts import PromptTemplate

template = PromptTemplate(
    input_variables=["name", "topic"],
    template="You are a helpful assistant. \
    Explain {topic} to {name} in simple words."
)

prompt = template.format(name="Nikhil",
                         topic="LangChain")
print(prompt)
```

You can define multiple variables in a single template.

5. ChatPromptTemplate (for Chat Models)

```
from langchain.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful AI assistant."),
    ("user", "Explain {topic} in simple words.")
])

prompt = template.format_messages(topic="MLOps")
print(prompt)
```

ChatPromptTemplate allows you to define system, user and assistant messages.

6. Few Shot Prompting (Example)

```
from langchain.prompts import FewShotPromptTemplate
examples = [
    {"input": "2 + 2", "output": "4"},
    {"input": "5 + 3", "output": "8"}
]
template = "Question: {input}\nAnswer:"
prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=PromptTemplate(
        input_variables=["input", "output"],
        template="Question: {input}\nAnswer: {output}"
    ),
    suffix=template,
    input_variables=["input"]
)
print(prompt.format(input="10 + 5"))
```

Few shot prompting gives examples before the actual question.

7. Best Practices for Prompting

- Be clear and specific
- Provide context
- Use role based instructions
- Break complex tasks into smaller steps
- Use examples (few shot) if needed
- Control output format (JSON, bullet points, etc.)
- Set constraints (length, style, tone, etc.)
- Iterate and refine your prompts

"Good prompts save time and reduce unnecessary iterations."

8. Common Prompt Templates

Template	Use Case
Summarization	Summarize long text
Q&A	Answer questions from context
Code Generation	Generate code from description
Data Extraction	Extract structured data (JSON)
Explanation	Explain complex concepts
Translation	Translate text to another language
Creative Writing	Generate blogs, stories, etc.
Classification	Classify text into categories

You can create custom templates for your specific use case.

9. Key Takeaways

- Prompts are the input to LLMs
- Prompt templates make prompts reusable
- Use ChatPromptTemplate for chat models
- Try different prompting techniques (zero shot, few shot, chain of thought)
- Good prompting improves output quality
- Experiment and iterate for better results

"Prompting is a skill.
The better you get at it,
the more powerful AI becomes."

Learn Practice Build Real Projects !

Same Concepts Bigger Impact !

Better Conversations Better Applications !

5. Messages + Message History

@codedsoul_05

Learn Build Apply Grow !

1. What are Messages ?

- Messages are the standard format to communicate with chat models in LangChain.
- They represent the role and content of each message in a conversation.
- Each message has a role (system, user, assistant, tool, etc.) and content (text, image, etc.).
- LangChain uses message objects instead of plain strings for better flexibility and control.

"Messages make conversations structured, consistent and powerful."

2. Types of Messages

Message Type	Class	Use Case
System Message	SystemMessage	Set behavior and instructions
Human Message	HumanMessage	User input
AI Message	AIMessage	Model response
Tool Message	ToolMessage	Response from a tool
Function Message	FunctionMessage	Function calling responses
Chat Message	ChatMessage	Custom role messages (if needed)

Different message types help in building structured and multi turn conversations.

3. Creating Messages (Examples)

```
from langchain.schema import (
    SystemMessage,
    HumanMessage,
    AIMessage,
    ToolMessage
)
messages = [
    SystemMessage(content="You are a helpful
    AI assistant."),
    HumanMessage(content="What is MLOps?"),
    AIMessage(content="MLOps stands for
      Machine Learning Operations..."),
]
```

Use message objects to build conversations with roles and structure.

4. Passing Messages to a Chat Model

```
from langchain.chat_models import ChatOpenAI
from langchain.schema import HumanMessage,
     SystemMessage

chat = ChatOpenAI(model="gpt-3.5-turbo")

messages = [
  SystemMessage(content="You are a helpful assistant."),
  HumanMessage(content="Explain LangChain in simple words.")
]

response = chat.invoke(messages)
print(response.content)
```

Pass a list of message objects to the model instead of a single string.

5. ChatMessage (Custom Role)

```
from langchain.schema import ChatMessage

messages = [
   ChatMessage(role="developer",
        content="You are an expert in AI."),
   ChatMessage(role="user",
        content="Give me best resources")
]

chat = ChatOpenAI(model="gpt-3.5-turbo")
response = chat.invoke(messages)
print(response.content)
```

Use ChatMessage to define custom roles beyond system, human and assistant.

6. Message History (In Memory)

```
from langchain.memory import ChatMessageHistory

history = ChatMessageHistory()

history.add_user_message("Hello!")
history.add_ai_message("Hi! How can I help you?")
history.add_user_message("Tell me about MLOps.")

# Get all messages
for msg in history.messages:
   print(f"{msg.type}: {msg.content}")
```

ChatMessageHistory stores conversation messages in memory.

7. Using Message History with Chain

```
from langchain.chains import ConversationChain
from langchain.chat_models import ChatOpenAI
from langchain.memory import ChatMessageHistory

chat = ChatOpenAI(model="gpt-3.5-turbo")
memory = ChatMessageHistory()

chain = ConversationChain(
    llm=chat,
    memory=memory,
    verbose=True
)
chain.run("Hi, my name is Nikhil.")
chain.run("What is my name?")
```

ConversationChain automatically uses message history.

8. Other Memory Options

Memory Type	Description
ChatMessageHistory	In memory message storage (for manual use)
ConversationBufferMemory	Stores all conversation (history as text)
ConversationBufferWindowMemory	Stores last k messages
ConversationSummaryMemory	Summarizes old messages
ConversationSummaryBufferMemory	Hybrid of summary + buffer
VectorStoreRetrieverMemory	Stores messages in a vector store (long term)

Choose the right memory based on your use case and conversation length.

9. Key Takeaways

- Messages are the core format for chat models in LangChain
- Use SystemMessage, HumanMessage, AIMessage, ToolMessage as per need
- ChatMessage allows custom roles
- Message history helps maintain context in multi turn conversations
- LangChain provides different memory classes for different use cases
- Structured messages lead to better, more reliable conversations
- Use message history for chatbots, agents and long term applications

"Good memory makes great conversations."

Build Real Projects Gain Real Skills !

Same Concepts Bigger Opportunities !

Chain
Build
Combine
Solve !

6. Chains + LCEL

@codedsoul_05

Same
Concepts
Bigger
Applications !

1. What are Chains ?

- Chains are sequences of components that are connected to perform a task.
- They combine multiple steps like prompting, calling a model, parsing output, etc.
- Chains help in building complex workflows in a simple and modular way.
- In LangChain, chains can be created using traditional Chain classes or the newer LCEL.

"Chains turn multiple steps into one simple pipeline."

2. Why LCEL ?

- LCEL (LangChain Expression Language) is a modern, simpler and more powerful way to build chains.
- It uses a pipe operator (|) to connect components.
- More readable and easier to debug.
- Supports streaming, parallel execution and complex workflows.
- Works with all LangChain components (prompts, models, parsers, retrievers, etc.).

"LCEL makes building chains as easy as writing a simple expression."

3. Basic LCEL Chain (Example)

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template(
    """You are a helpful assistant.
    Explain {topic} in simple words."""
)
model = ChatOpenAI(model="gpt-3.5-turbo")
parser = StrOutputParser()

chain = prompt | model | parser

response = chain.invoke({"topic": "LangChain"})
print(response)
```

Use | to connect components in LCEL.

4. Chain Components

Component	Description
PromptTemplate	Creates a prompt from a template
ChatModel	LLM to generate response
OutputParser	Parses and formats model output
Retriever	Fetches relevant documents
Tool	External tool or API
Runnable	Any component that can be run
Lambda	Custom function in a chain
Passthrough	Passes input as is

These components can be combined using LCEL.

5. Different Chain Examples

a) Simple LLM Chain

```
chain = prompt | model | parser
# prompt -> model -> parse output
```

b) Chain with Retriever (RAG)

```
chain = (
  {"context": retriever, "question": RunnablePassthrough()}
  | prompt
  | model
) | parser
```

c) Chain with Custom Function

```
chain = (
  prompt
  | model
  | (lambda x: x.upper())   # custom step
) | parser
```

6. LCEL Features

- Composable: Connect components with |
- Readable: Clean and intuitive syntax
- Flexible: Supports branching, parallel execution and custom functions
- Streaming: Easy to enable streaming
- Async Support: Works with async/await
- Integration: Works with all LangChain tools, retrievers and models
- Debuggable: Use .with_config() and LangSmith
- Extensible: Create custom runnables

LCEL gives more control with less code.

7. Advanced LCEL Patterns

a) Parallel Execution

```
chain = {
  "summary": prompt1 | model | parser,
  "notes": prompt2 | model | parser
}
```

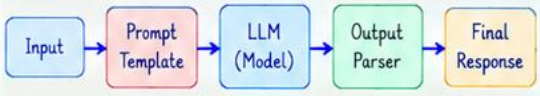
b) Conditional Chain (RunnableBranch)

```
from langchain_core.runnables import RunnableBranch

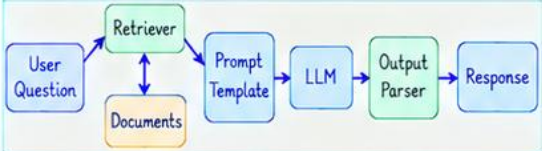
chain = RunnableBranch(
  (lambda x: "math" in x["topic"], math_chain),
  (lambda x: True, general_chain),
)
```

LCEL supports complex workflows with minimal code.

8. Chain Flow Diagram



RAG Chain Flow



Chains help connect multiple components to solve real world problems.

9. Key Takeaways

- Chains help in building multi step workflows.
- LCEL is the recommended way to build chains.
- Use | to connect components.
- You can combine prompts, models, retrievers, parsers, tools and custom functions.
- LCEL supports advanced patterns like parallel execution, branching and streaming.
- Start with simple chains and gradually explore complex workflows.

"Simple chains today, intelligent applications tomorrow."

More
Chains
More Possibilities !

Same
Concepts
Bigger
Impact !

Structured Output Better Applications !

7. Output Parsers + Structured Output

@codedsoul_05

Same Concepts Bigger Impact !

1. What are Output Parsers ?

- Output parsers convert the raw text output from an LLM into a structured format (like JSON, Pydantic model, list, etc.).
- They help in getting consistent and reliable outputs.
- Useful when you want to integrate LLM output with your application logic.
- LangChain provides many built-in parsers and also allows custom parsers.

"Output parsers turn unstructured text into usable data."

2. Why Structured Output ?

- LLMs can generate different formats each time.
- Structured output ensures consistency.
- Helps in automation and integration with other systems.
- Useful for APIs, databases, agents and production applications.
- Reduces the need for complex post-processing.

"Structured output = Reliable, Consistency, Automation."

3. Common Output Parsers in LangChain

Parser	Description
StrOutputParser	Returns output as plain string
CommaSeparatedListOutputParser	Parses comma separated list
JsonOutputParser	Parses JSON output
PydanticOutputParser	Parses output into a Pydantic model
StructuredOutputParser	Parses based on defined schema
EnumOutputParser	Parses output into an enum value
DatetimeOutputParser	Parses date and time values
RetryOutputParser	Retries parsing on failure
OutputFixingParser	Fixes and parses invalid output

Choose the right parser based on your use case.

4. StrOutputParser (Simple Example)

```
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

model = ChatOpenAI(model="gpt-3.5-turbo")
parser = StrOutputParser()

chain = model | parser

response = chain.invoke("What is MLOps?")
print(response)
```

Returns output as a simple string.

5. JsonOutputParser (JSON Output)

```
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import JsonOutputParser

model = ChatOpenAI(model="gpt-3.5-turbo")
parser = JsonOutputParser()

chain = model | parser

response = chain.invoke(
    """Return a JSON with keys: name, age, role
       for a software engineer."""
)
print(response)
```

Parses the output into a Python dictionary (JSON format).

6. PydanticOutputParser (Typed Output)

```
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import PydanticOutputParser

class Person(BaseModel):
    name: str = Field(description="Name of the person")
    age: int = Field(description="Age of the person")
    role: str = Field(description="Role of the person")

parser = PydanticOutputParser(pydantic_object=Person)
model = ChatOpenAI(model="gpt-3.5-turbo")

chain = model | parser

response = chain.invoke("Tell me about a software engineer.")
print(response)
```

Parses the output into a Pydantic model (with validation).

7. StructuredOutputParser (Custom Schema)

```
from langchain.output_parsers import StructuredOutputParser,
    ResponseSchema

response_schemas = [
    ResponseSchema(name="name", description="Name of person"),
    ResponseSchema(name="skills", description="List of skills"),
    ResponseSchema(name="experience", description="Years of
    experience"),
]
parser = StructuredOutputParser.from_response_schemas(
    response_schemas
)
format_instructions = parser.get_format_instructions()

model = ChatOpenAI(model="gpt-3.5-turbo")

chain = model | parser

response = chain.invoke(
    """{format_instructions}
     Tell me about a data scientist."""
)
print(response)
```

Define a custom schema for structured output.

8. OutputFixingParser (Handle Invalid Output)

```
from langchain.output_parsers import OutputFixingParser, JsonOutputParser
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-3.5-turbo")
base_parser = JsonOutputParser()
parser = OutputFixingParser.from_llm(parser=base_parser, llm=model)

chain = model | parser

response = chain.invoke("Return a JSON with keys: topic, points")
print(response)
```

Automatically fixes common formatting errors and parses the output.

9. Key Takeaways

- Output parsers convert LLM output into structured data.
- Use the right parser based on your need.
- Structured output is essential for real world applications.
- PydanticOutputParser is great for validation.
- StructuredOutputParser gives flexibility with custom schema.
- OutputFixingParser helps handle parsing errors.
- Structured output enables reliable, consistent and automated workflows.

Real World Use Cases

- ✓ Extracting information from documents
- ✓ Building APIs with LLMs
- ✓ Storing data in databases
- ✓ Agent tool outputs
- ✓ Form filling and data extraction
- ✓ Automation and reporting
- ✓ Integrating with external systems

Pro Tips

- Always give clear format instructions.
- Use Pydantic models for type safety.
- Handle parsing errors gracefully.
- Test with different inputs.
- Use OutputFixingParser for reliability.
- Combine parsers with retry logic if needed.

"Structured output today, Smarter applications tomorrow."

Parse Structure Build Scale !

Same Concepts Bigger Opportunities !

Turn Unstructured Data Into Knowledge !

8. Embeddings + Vector Stores

@codedsoul_05

Learn Build Apply Grow !

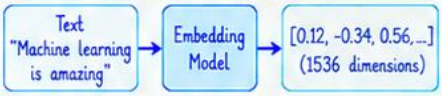
1. What are Embeddings ?

- Embeddings convert text, images or other data into high dimensional numerical vectors.
- These vectors capture the meaning and semantic similarity of the data.
- Similar content gets similar vectors.
- Embeddings allow machines to understand and compare content beyond exact keyword matching.

"Embeddings turn words into numbers that capture meaning."

2. How Embeddings Work ?

- A model (like OpenAI, HuggingFace, etc.) converts text into a vector of numbers.
- Text with similar meaning has vectors closer to each other in vector space.
- These vectors can be compared using similarity metrics like cosine similarity.
- Useful for search, retrieval, clustering and recommendation systems.



Similar meaning → Similar vectors
Different meaning → Distant vectors

3. Generating Embeddings (Example)

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small"
)

text = "Machine learning is amazing"
vector = embeddings.embed_query(text)

print(len(vector))   # 1536
print(vector[:5])    # first 5 values
```

You can also use HuggingFaceEmbeddings, CohereEmbeddings, etc.

4. Common Embedding Models

Provider	Model Name	Dimension	Use Case
OpenAI	text-embedding-3-small	1536	General use
OpenAI	text-embedding-3-large	3072	High quality
HuggingFace	all-MiniLM-L6-v2	384	Lightweight
Cohere	embed-english-v3	1024	Multilingual
Google	text-embedding-004	768	Multimodal
Instructor	instructor-large	768	Better context
BGE	bge-large-en	1024	High performance

Choose a model based on your use case, accuracy needs and cost.

5. What are Vector Stores ?

- Vector stores store embedding vectors along with the original data (text, metadata).
- They allow fast and efficient similarity search on large datasets.
- Used in Retrieval Augmented Generation (RAG) to find relevant context from documents.
- Popular vector stores: FAISS, Chroma, Pinecone, Weaviate, Qdrant, Milvus, etc.

"Vector stores are like databases for embeddings, optimized for similarity search."

6. Using a Vector Store (Chroma Example)

```
from langchain_community.document_loaders import
    TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma

loader = TextLoader("data.txt")
docs = loader.load()

splitter = RecursiveCharacterTextSplitter(chunk_size=500,
                                          chunk_overlap=50)
chunks = splitter.split_documents(docs)

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db"
)

print("Documents stored in Chroma!")
```

This loads documents, splits them, creates embeddings and stores them in Chroma.

7. Similarity Search (Retrieve Data)

```
query = "What is machine learning?"

results = vectorstore.similarity_search(query, k=3)

for doc in results:
    print(doc.page_content)
    print("---")
```

Returns the most relevant documents based on semantic similarity.

8. Vector Similarity Metrics

Metric	Description	Use Case
Cosine Similarity	Measures angle between vectors	Most common (for text)
Euclidean Distance	Straight line distance	Numerical data
Dot Product	Measures alignment	Used in some models
Manhattan Distance	Sum of absolute differences	Less common

Cosine similarity is most commonly used for text embeddings.

9. Real World Applications

- Semantic search (docs, websites, FAQs)
- RAG for chatbots and AI assistants
- Recommendation systems
- Clustering and classification
- Image and video search (multimodal)
- Enterprise knowledge search
- Personalized content and ads

10. Key Takeaways

- Embeddings convert data into numerical vectors.
- Vector stores enable fast similarity search.
- Used heavily in RAG and real world AI systems.
- Choose the right embedding model and vector store based on your use case.
- Understand similarity metrics.
- Embeddings + vector stores unlock powerful applications.

"Embeddings give meaning to data, Vector stores give it memory."

Store Search Retrieve Generate !

Same Concepts Bigger Impact !

Load Process Split Use !

9. Document Loaders + Text Splitters

@codedsoul_05

Same Concepts Bigger Opportunities !

1. What are Document Loaders ?

- Document loaders help you load data from different sources (PDF, TXT, Web, Notion, YouTube, etc.) into LangChain.
- They convert the data into a standard format called "Document" objects.
- Each Document contains page_content (the text) and metadata (source, page number, etc.).
- LangChain provides many built-in loaders for different file types and platforms.

"Document loaders bring your data into LangChain."

2. Document Object Structure

```
from langchain_core.documents import Document

doc = Document(
    page_content="This is a sample text.",
    metadata={
        "source": "example.pdf",
        "page": 1
    }
)

print(doc.page_content)
print(doc.metadata)
```

A Document = text + metadata

3. Common Document Loaders

Loader	Use Case	Import Class
TextLoader	Load .txt files	TextLoader
PyPDFLoader	Load PDF files	PyPDFLoader
CSVLoader	Load CSV files	CSVLoader
WebBaseLoader	Load web pages (URLs)	WebBaseLoader
UnstructuredFileLoader	Load various file types	UnstructuredFileLoader
NotionLoader	Load from Notion	NotionLoader
YouTubeLoader	Load YouTube transcripts	YoutubeLoader
DirectoryLoader	Load multiple files from a folder	DirectoryLoader

Choose the right loader based on your data source.

4. Example: Load a PDF Document

```
from langchain_community.document_loaders import
    PyPDFLoader

loader = PyPDFLoader("data/notes.pdf")
docs = loader.load()

print(len(docs))                    # number of pages
print(docs[0].page_content[:200]) # first 200 chars
print(docs[0].metadata)
```

Returns a list of Document objects, one per page.

5. Example: Load a Web Page

```
from langchain_community.document_loaders import
    WebBaseLoader

loader = WebBaseLoader("https://example.com")
docs = loader.load()

print(len(docs))
print(docs[0].page_content[:200])
```

Useful for loading blog posts, documentation and any public webpage.

6. Example: Load Multiple Files

```
from langchain_community.document_loaders import
    DirectoryLoader, PyPDFLoader

loader = DirectoryLoader(
        "data/",
        glob="*.pdf",
        loader_cls=PyPDFLoader
    )
docs = loader.load()
print(f"Total documents: {len(docs)}")
```

Loads all PDF files from the specified folder.

7. What are Text Splitters ?

- Text splitters divide large documents into smaller chunks.
- LLMs have a limited context window, so we split text to fit within the limit.
- Splitting helps in better retrieval and more relevant responses.
- LangChain provides different splitting strategies based on your data type.

Split large text into smaller, meaningful chunks.

8. Common Text Splitters

Splitter	Use Case	Import Class
CharacterTextSplitter	Simple character based splitting	CharacterTextSplitter
RecursiveCharacter TextSplitter	Splits using hierarchy (char, then sentence, then word)	RecursiveCharacter TextSplitter
TokenTextSplitter	Split based on token count (using tiktoken)	TokenTextSplitter
SentenceTransformers TokenTextSplitter	Token based splitting (with sentence transformers)	SentenceTransformers TokenTextSplitter
MarkdownTextSplitter	Good for markdown files	MarkdownTextSplitter
HTMLTextSplitter	Good for HTML content	HTMLTextSplitter

RecursiveCharacterTextSplitter is most commonly used.

9. Example: Split Text into Chunks

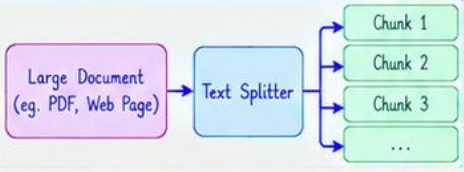
```
from langchain.text_splitter import
    RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    length_function=len,
)
chunks = text_splitter.split_documents(docs)

print(f"Total chunks: {len(chunks)}")
print(chunks[0].page_content[:200])
```

chunk_size = size of each chunk
chunk_overlap = overlapping text between chunks

10. Chunking Strategies (Visual)



Split the document into smaller chunks for better retrieval and processing.

11. Best Practices

- Choose the right loader for your data source.
- Clean and preprocess text if needed.
- Use RecursiveCharacterTextSplitter for most cases.
- Set appropriate chunk_size and chunk_overlap.
- Maintain metadata (source, page number, etc.).
- Experiment with different splitters for best results.
- Test chunk quality with your use case (RAG, agents, etc.).

Good splitting = Better retrieval = Better answers.

12. Key Takeaways

- Document loaders help you bring external data into LangChain.
- Text splitters break large text into smaller chunks for LLMs.
- Use the right loader and splitter for your use case.
- Maintain metadata for better traceability.
- Proper chunking improves retrieval quality and final responses.
- Experiment and customize as needed.

Load the right data. Split it smartly. Build something amazing."

More Data More Knowledge !

Same Concepts Bigger Impact !

Find Relevant Information Generate Better Answers !

10. Retrievers + RAG

@codedsoul_05

Learn Build Apply Grow !

1. What are Retrievers ?

- Retrievers are components that search and fetch relevant documents from a data source based on a user query.
- They convert the query into embeddings and find the most similar documents.
- Used in RAG (Retrieval Augmented Generation) to provide context to the LLM.
- Common retrievers: Vector store retrievers, BM25 retrievers, hybrid retrievers, etc.

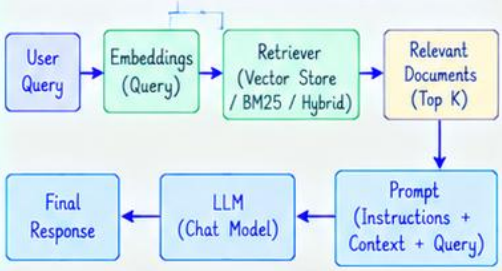
"Retrievers help LLMs find the right information instead of relying only on pretrained knowledge."

2. What is RAG ?

- RAG (Retrieval Augmented Generation) combines retrieval and generation.
- It first retrieves relevant documents from your data and then passes them to an LLM to generate a grounded answer.
- Useful for domain specific knowledge, chatbots, documentation, and real time data.
- Reduces hallucinations and provides more accurate and up to date responses.

"RAG = Retrieve relevant data + Generate a better answer."

3. RAG Architecture (High Level)



Retrieve relevant context and let the LLM generate a grounded and accurate answer.

4. Types of Retrievers

Retriever Type	Description	Use Case
Vector Store Retriever	Uses embeddings and similarity search	Most common for RAG
BM25 Retriever	Keyword based lexical search	Good for exact matches
Hybrid Retriever	Combines vector and keyword search	Better recall and precision
Multi Query Retriever	Generates multiple queries for better recall	Complex queries
Self Query Retriever	Uses LLM to create structured queries	Metadata filtering

Choose the right retriever based on your data and user query type.

5. Example: Vector Store Retriever

```python
from langchain_community.document_loaders import TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

# Load and split documents
loader = TextLaader("data.txt")
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=500,
                                          chunk_overlap=50)
split_docs = splitter.split_documents(docs)

# Create vector store
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = Chroma.from_documents(split_docs, embeddings)
retriever = vectorstore.as_retriever(search_type="similarity",
                                     search_kwargs={"k": 4})
```

Create a vector store and use it as a retriever to fetch similar documents.

6. Example: RAG Chain

```python
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-3.5-turbo")

qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    chain_type="stuff"
)

response = qa_chain.invoke(
  {"query": "What is machine learning?"}
)

print(response["result"])
```

Combine retriever and LLM to build a simple RAG pipeline.

7. Advanced RAG Techniques

- Hybrid Search (vector + keyword)
- Re-ranking (improves relevance of results)
- Query Expansion (generate multiple queries)
- Self Query Retriever (LLM powered filtering)
- Metadata Filtering (filter by source, date, etc.)
- Context Compression (remove irrelevant content)
- Multi hop Retrieval (for complex questions)
- Agentic RAG (use agents for tool based retrieval)

Use advanced techniques to improve retrieval quality and answer accuracy.

8. Evaluation Metrics

Metric	Description	Use Case
Recall	How many relevant documents were retrieved	Check coverage
Precision	How many retrieved docs are actually relevant	Check relevance
MRR	Mean Reciprocal Rank	Best match position
Hit Rate	Whether relevant doc is in top K	Simple evaluation
Answer Relevance	Quality of final LLM answer	End to end check

Evaluate both retrieval and final answer for better RAG systems.

9. Real World Use Cases

- Chatbots on company documentation
- Q&A on PDFs, websites, and knowledge bases
- Customer support bots
- Research assistants (papers, articles)
- Enterprise search across internal data
- Legal, medical, and finance domain applications
- Personal knowledge management (notes, docs)

10. Key Takeaways

- Retrievers fetch relevant information from your data.
- RAG combines retrieval and generation for better and grounded answers.
- Choose the right retriever and use advanced techniques for better results.
- Evaluate your system using proper metrics.
- RAG reduces hallucinations and enables real world applications.

Right information + Right model = Real impact.

Retrieve Augment Generate Solve !

Better LLMs with your Own Data !

Give LLM Superpowers with Tools !

11. Tools + Tool Calling

@codedsoul_05

Same Concepts Bigger Possibilities !

1. What are Tools ?

- Tools are external functions, APIs or services that LLMs can use to perform actions beyond just generating text.
- They allow the model to interact with the real world (search the web, get live data, run code, query databases, etc.).
- In LangChain, tools are defined with a name, description and a function to execute.
- Tools make LLMs more useful, accurate and up to date.

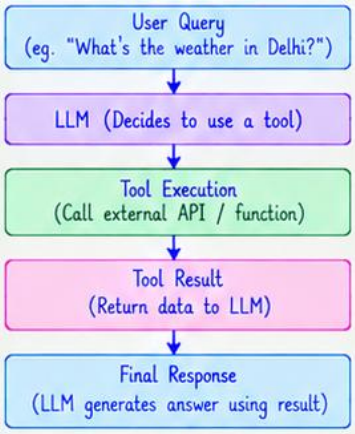
Tools turn LLMs from a text generator into an action taker.

2. Why Tool Calling ?

- LLMs alone don't have real time information or access to external systems.
- Tool calling allows the model to decide when to use a tool to get the required information.
- Helps in building more reliable and factual applications.
- Useful for tasks like search, calculations, booking, database queries, automation etc.

Tool calling = LLM decides + calls the right tool + gets the result + gives a final answer.

3. How Tool Calling Works ?



4. Defining a Tool (Example)

```python
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    """Get current weather for a city."""
    # In real case, call a weather API
    return f"The current weather in {city} is sunny with 32°C."
```

A tool has a name, description and a function that returns a result.

5. Using Tools with an LLM

```python
from langchain_openai import ChatOpenAI
from langchain.agents import initialize_agent, Tool
llm = ChatOpenAI(model="gpt-3.5-turbo")

tools = [get_weather]  # list of tools

agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent="zero-shot-react-description",
    verbose=True
)

response = agent.invoke("What's the weather in Delhi?")
print(response)
```

Agent uses the tool when needed and returns the final answer.

6. Types of Tools

Tool Type	Description	Example
API Tool	Calls external APIs	Weather API, Google Maps
Search Tool	Search information from the web	Tavily, SerpAPI
Database Tool	Query databases	SQL Database
Python REPL	Execute Python code	Calculations, data analysis
File Tool	Read / write files	Local file system
Custom Tool	Your own function	Business logic

You can create custom tools for your own use case.

7. Tool Calling with Chat Models

```python
from langchain_openai import ChatOpenAI
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

llm = ChatOpenAI(model="gpt-3.5-turbo")
response = llm.bind_tools([add]).invoke(
    "Add 25 and 17 using the tool"
)
print(response)
```

Chat models can call tools directly using bind_tools().

8. Agent vs Direct Tool Calling

Feature	Direct Tool Calling	Agents
Control	You decide when to call	LLM decides when to call
Flexibility	Simple and lightweight	Handles complex workflows
Use Case	Specific, single task	Multi step tasks with reasoning
Setup	Easier	Slightly complex
Best For	Simple integrations	Real world applications

Use direct tool calling for simple use cases and agents for complex workflows.

9. Best Practices

- Write clear and descriptive tool descriptions.
- Keep tool outputs concise and structured.
- Handle errors and invalid inputs gracefully.
- Limit the number of tools to avoid confusion.
- Use caching where possible to reduce API calls.
- Validate and sanitize tool inputs.
- Monitor tool usage and costs.

10. Key Takeaways

- Tools give LLMs real world capabilities.
- LangChain makes it easy to define and use tools.
- Tool calling improves accuracy and usefulness of LLM applications.
- You can use built in tools or create your own.
- Use agents for complex multi step tasks.
- Choose the right tool and design it well.

LLMs + Tools = Real World Impact.

More Tools More Impact !

Same Concepts Bigger Applications !

Plan
Reason
Take Action
Repeat
Get Results !

12. Agents + Agent Loops

@codedsoul_05

Same Concepts Bigger Possibilities !

1. What are Agents ?

- Agents are AI systems that can understand a goal, plan steps, use tools and take actions to complete the task.
- Unlike simple LLM calls, agents can decide what to do next.
- They can interact with external tools, APIs, databases, and the real world.
- LangChain provides a framework to build agents easily.

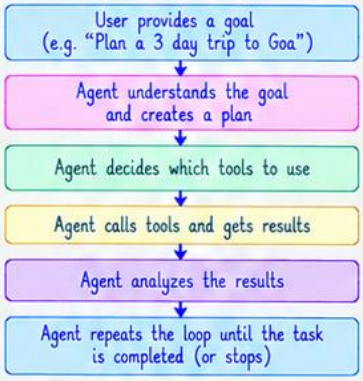
"Agents turn LLMs from passive answer machines into active problem solvers."

2. Why Agents ?

- Real world problems often need multiple steps, not just a single response.
- Agents can break complex tasks into smaller steps and execute them.
- They can use tools like search, calculator, file system, database, or custom APIs.
- Useful for automation, research, booking, data analysis, and many real world applications.

"Agents help you get things done, not just answers."

3. How Agents Work ?



Plan → Act → Observe → Repeat → Final Answer

4. Basic Agent Example

```
from langchain.agents import initialize_agent, Tool
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-3.5-turbo")

tools = [
    Tool(
        name="Search",
        func=search_tool.run,
        description="Search the web for information"
    )
]

agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent="zero-shot-react-description",
    verbose=True
)

response = agent.invoke("Find latest news about GATE 2027")
print(response)
```

A simple agent that can use a search tool to answer a question.

5. Types of Agents in LangChain

Agent Type	Description	Use Case
Zero Shot ReAct	Decides tools based on description	General purpose tasks
Structured Chat	Works with multiple tools and structured input	Complex tool usage
OpenAI Functions Agent	Uses OpenAI function calling	When using function calling models
Plan and Execute	First creates a plan then executes	Long and complex tasks
Self Ask with Search	Breaks question into sub questions	Research and information gathering
Conversational Agent	Maintains chat history	Chatbots with tools

Choose the right agent type based on your task and complexity.

6. Agent Loop (ReAct Pattern)

1	Thought	Agent thinks about what to do
2	Action	Agent calls a tool
3	Observation	Agent gets the result from the tool
4	Repeat	Agent decides next step
5	Finish	Agent returns the final answer

Example Loop (Simplified)

Thought: I need to search for the latest GATE 2027 date.
Action: Use search tool.
Observation: Got the result.
Thought: Now summarize the information.
Action: Give final answer.
Final Answer: GATE 2027 will be conducted in February 2027.

Agent loops help the model reason, take actions and improve step by step.

7. Memory in Agents

- Agents can use memory to remember previous interactions.
- Useful for multi turn conversations and long tasks.
- LangChain provides different memory types like ConversationBufferMemory, ConversationSummaryMemory, etc.

```
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory()
agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent="conversational-react-description",
    memory=memory,
    verbose=True
)
```

Memory helps agents have meaningful longer conversations.

8. Custom Tools

- You can create your own tools for specific use cases.
- Tools can be Python functions, APIs, database queries, or any custom logic.

```
from langchain.tools import tool

@tool
def get_stock_price(symbol: str) -> str:
    """Get the current stock price."""
    # your logic here
    return f"The current price of {symbol} is ₹2500."

# Use in agent
tools = [get_stock_price]
```

Custom tools make your agents more powerful and useful.

9. Multi Agent Systems

- You can have multiple agents working together.
- Each agent can have a specific role (e.g. researcher, planner, executor).
- Useful for complex workflows.



Multiple agents can collaborate to solve bigger problems.

10. Key Takeaways

- Agents can plan, use tools and complete complex tasks.
- Agent loops follow a plan act observe repeat pattern.
- LangChain provides many types of agents.
- You can use memory and custom tools to make agents more powerful.
- Multi agent systems are useful for real world applications.
- Agents are the next step towards more autonomous AI systems.

"Agents don't just answer questions, they make things happen."

More Agents More Solutions !

Same Concepts Bigger Impact !

Remember Context Build Better Conversations !

13. Memory + Conversation History

@codedsoul_05

Learn Build Apply Grow !

1. What is Memory ?

- Memory allows an LLM to remember information from previous interactions.
- It helps maintain context across multiple turns in a conversation.
- Without memory, each chat is independent and the model forgets earlier messages.
- Memory can be short term (within a session) or long term (across sessions).
- Useful for chatbots, assistants, and personalized AI applications.

"Memory turns a single response model into a real conversational AI."

2. Why Memory is Important ?

- Maintains context and continuity.
- Gives more relevant and accurate responses.
- Enables natural, human like conversations.
- Useful for tasks like customer support, personal assistants, tutoring, etc.
- Reduces the need to repeat information again and again.
- Improves user experience and trust.

"Good memory = Better context = More helpful and natural conversations."

3. Types of Memory in LangChain

Memory Type	Description	Use Case
ConversationBufferMemory	Stores all messages in memory	Simple chatbots
ConversationBufferWindowMemory	Stores last k messages	Limit context size
ConversationSummaryMemory	Summarizes old messages	Long conversations
ConversationSummaryBufferMemory	Combines buffer + summarization	Balanced approach
EntityMemory	Remembers key entities (names, facts)	Personalized chats
VectorStoreRetrieverMemory	Stores past chats in a vector store	Long term memory (across sessions)

Choose the right memory type based on your use case and conversation length.

4. Basic Example (ConversationBufferMemory)

```python
from langchain_openai import ChatOpenAI
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain

llm = ChatOpenAI(model="gpt-3.5-turbo")
memory = ConversationBufferMemory()
conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

conversation.run("Hi, my name is Nikhil.")
conversation.run("What is my name?")  # Remembers!
```

The model remembers your previous messages in the same session.

5. Using a Window (Last k Messages)

```python
from langchain_openai import ChatOpenAI
from langchain.memory import ConversationBufferWindowMemory
from langchain.chains import ConversationChain

llm = ChatOpenAI(model="gpt-3.5-turbo")
memory = ConversationBufferWindowMemory(k=3)
conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)
conversation.run("Message 1")
conversation.run("Message 2")
conversation.run("Message 3")
conversation.run("Message 4")  # Only last 3 are kept
```

Keeps only the last k messages to manage token usage.

6. Using Summary Memory (Long Chats)

```python
from langchain_openai import ChatOpenAI
from langchain.memory import ConversationSummaryMemory
from langchain.chains import ConversationChain

llm = ChatOpenAI(model="gpt-3.5-turbo")
memory = ConversationSummaryMemory(
    llm=llm
)
conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)
conversation.run("Explain what is machine learning.")
conversation.run("Now summarize what we discussed so far.")
```

Summarizes older messages to keep important context and save tokens.

7. Entity Memory (Remember Key Facts)

```python
from langchain_openai import ChatOpenAI
from langchain.memory import ConversationEntityMemory
from langchain.chains import ConversationChain

llm = ChatOpenAI(model="gpt-3.5-turbo")
memory = ConversationEntityMemory(llm=llm)
conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

conversation.run("My name is Nikhil and I live in Ahmedabad.")
conversation.run("Where do I live?")  # Remembers entity
```

Automatically extracts and remembers important entities like names, places, and facts.

8. Storing Conversation History

```python
from langchain.memory import ChatMessageHistory
from langchain.memory import ConversationBufferMemory

history = ChatMessageHistory()
history.add_user_message("Hi, I am Nikhil.")
history.add_ai_message("Hello Nikhil!")

memory = ConversationBufferMemory(
    chat_memory=history,
    return_messages=True
)

print(memory.load_memory_variables({}))
```

You can store, retrieve and manage conversation history manually.

9. Best Practices

- Choose the right memory type for your use case.
- Use summarization for long conversations.
- Limit the context window to avoid high token usage.
- Persist memory in a database for long term use (e.g. Redis, MongoDB, or Vector Store).
- Combine memory with retrieval (RAG) for better results.
- Handle sensitive data and provide option to clear chat history.
- Test how memory affects response quality.

10. Key Takeaways

- Memory helps LLMs maintain context across turns.
- Different memory types solve different problems.
- It improves conversation quality and personalization.
- Use summarization and external storage for long term memory.
- A well designed memory system makes your AI application more useful and human like.

"Remember the past. Build a better future."

Context today Smarter AI tomorrow !

Same Concepts Bigger Possibilities !

See What's Happening Behind the Scenes !

14. Callbacks + Streaming

@codedsoul_05

Same Concepts Bigger Possibilities !

1. What are Callbacks ?

- Callbacks are functions that get called during the execution of a LangChain application.
- They help you track what is happening inside the LLM pipeline (start, end, error, intermediate steps, etc.).
- Useful for logging, monitoring, debugging and custom actions.
- LangChain provides a callback system to hook into different events.

"Callbacks give you visibility and control over the LLM execution flow."

2. What is Streaming ?

- Streaming means getting the LLM response token by token instead of waiting for the full response.
- It reduces perceived latency and improves user experience.
- Useful for chat applications, long responses and real time interfaces.
- LangChain supports streaming out of the box with callbacks or direct streaming methods.

"Streaming makes AI responses feel faster and more interactive."

3. Callback Events in LangChain

Event	When it is Called	Use Case
on_llm_start	When LLM starts processing	Log input, start timer
on_llm_new_token	When a new token is generated	Streaming, live display
on_llm_end	When LLM finishes	Log output, stop timer
on_chain_start	When a chain starts	Track chain execution
on_chain_end	When a chain ends	Get final result
on_tool_start	When a tool starts	Log tool usage
on_tool_end	When a tool ends	Log tool result
on_error	When an error occurs	Handle errors

Use callbacks to monitor, debug and customize the behavior of your application.

4. Simple Callback Example

```
from langchain.callbacks.base import BaseCallbackHandler
from langchain_openai import ChatOpenAI

class MyHandler(BaseCallbackHandler):
    def on_llm_start(self, serialized, prompts, **kwargs):
        print("LLM started...")

    def on_llm_new_token(self, token, **kwargs):
        print(f"New token: {token}")

    def on_llm_end(self, response, **kwargs):
        print("LLM finished.")

llm = ChatOpenAI(model="gpt-3.5-turbo",
                 callbacks=[MyHandler()])

llm.invoke("Explain LangChain in simple words.")
```

Callbacks let you hook into different stages of LLM execution.

5. Streaming Example

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-3.5-turbo",
                 streaming=True)

response = llm.invoke("Tell me a story about space.")
for chunk in response:
    print(chunk, end="", flush=True)

# Another way using callbacks
class StreamHandler(BaseCallbackHandler):
    def on_llm_new_token(self, token, **kwargs):
        print(token, end="", flush=True)

llm = ChatOpenAI(model="gpt-3.5-turbo",
                 callbacks=[StreamHandler()],
                 streaming=True)
llm.invoke("Explain quantum computing.")
```

Streaming gives a real time, token by token response.

6. Streaming with Chains

```
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate
from langchain_openai import ChatOpenAI

prompt = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic} in detail."
)

llm = ChatOpenAI(model="gpt-3.5-turbo",
                 streaming=True,
                 callbacks=[StreamHandler()])

chain = LLMChain(llm=llm, prompt=prompt)
chain.invoke({"topic": "Artificial Intelligence"})
```

Streaming also works with chains, agents and other LangChain components.

7. Practical Use Cases

- Display real time responses in chat applications
- Track token usage and cost
- Debug and log LLM behavior
- Show progress in long running tasks
- Monitor tool calls in agents
- Build better user experience with live output
- Integrate with logging tools (LangSmith, Console, Custom Logs)

Callbacks and streaming make your application more transparent and user friendly.

8. Tips and Best Practices

- Use callbacks for logging and monitoring
- Use streaming for better user experience
- Handle errors in callbacks
- Avoid heavy processing inside callback methods
- Use LangSmith for advanced tracing
- Combine callbacks with tools and agents
- Test with different models and providers

Keep it simple, light and focus on useful information.

9. Key Takeaways

- Callbacks allow you to track and customize the execution flow.
- Streaming provides real time, token by token responses.
- Both are useful for debugging, monitoring and building interactive applications.
- LangChain provides built in callback handlers and easy streaming support.
- Use them to build more reliable, transparent and user friendly AI applications.

"More visibility. Better control. Better AI apps."

Log Track Stream Improve !

Same Concepts Bigger Impact !

Build Stateful AI Agents Step by Step !

15. LangGraph Fundamentals

@codedsoul_05

Same Concepts Bigger Possibilities !

1. What is LangGraph ?

- LangGraph is a framework by LangChain for building stateful, multi step and controllable AI agents using graphs.
- It allows you to define workflows where nodes are functions (LLM calls, tools, logic) and edges control the flow.
- Unlike simple chains, LangGraph maintains state across steps and supports loops, branching and human in the loop.
- Useful for complex agents, multi agent systems and real world applications.

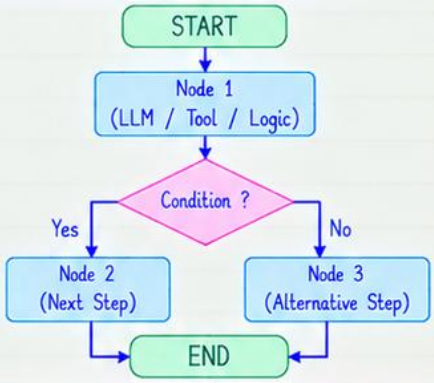
"LangGraph turns linear prompts into powerful, stateful workflows."

2. Key Concepts

- Graph: A collection of nodes and edges.
- Node: A function that takes the state, performs an action and returns an updated state.
- Edge: Defines which node to run next.
- State: Shared dictionary that stores data across the workflow.
- START and END: Special nodes to mark the beginning and end.
- Conditional Edges: Decide the next node based on state or output.

"Nodes do the work, edges decide the flow, state keeps the memory."

3. Basic Architecture



A graph defines how your agent thinks, acts and moves between steps.

4. Simple LangGraph Example

```python
from langgraph.graph import StateGraph, START, END
from typing import TypedDict
class AgentState(TypedDict):
    question: str
    answer: str

# Define nodes
def llm_node(state: AgentState):
    from langchain_openai import ChatOpenAI
    llm = ChatOpenAI(model="gpt-3.5-turbo")
    response = llm.invoke(state["question"])
    return {"answer": response.content}

# Build graph
graph = StateGraph(AgentState)
graph.add_node("llm", llm_node)
graph.add_edge(START, "llm")
graph.add_edge("llm", END)
app = graph.compile()
result = app.invoke({"question": "What is LangGraph?"})
print(result)
```

A minimal LangGraph with one node.

5. State Management

- State is a shared dictionary that flows through all nodes.
- Each node reads from the state, performs some action and returns an updated state.
- You can define custom state schemas using TypedDict, Pydantic or simple dict.
- State helps maintain context, track progress and pass data between steps.

```python
from typing import TypedDict

class AgentState(TypedDict):
    question: str
    answer: str
    steps: int

# Node updates state
def update_node(state: AgentState):
    return {"steps": state.get("steps", 0) + 1}
```

State keeps your agent organized and consistent.

6. Conditional Edges (Branching)

- Conditional edges allow the graph to decide the next node based on the current state.
- Useful for decision making, retries, tool usage and multi step reasoning.
- You define a function that returns the next node name based on logic.

```python
def decide_next(state: AgentState):
    if "error" in state:
        return "handle_error"
    elif state.get("steps", 0) < 3:
        return "continue"
    else:
        return "end"
graph.add_conditional_edges(
    "llm",
    decide_next,
    {"handle_error": "error_node",
     "continue": "llm",
     "end": END}
)
```

Branching makes your agent smarter and more flexible.

7. Loops and Iterations

- LangGraph supports loops to repeat tasks until a condition is met.
- Useful for retry mechanisms, self improvement and multi step reasoning.
- Use conditional edges to send the flow back to a previous node.

```python
def check_retry(state: AgentState):
    if state.get("steps", 0) < 3:
        return "llm" # loop back
    else:
        return END
```

Loops help your agent think, refine and get better results.

8. Human in the Loop

- You can add human input at any step in the graph.
- Useful for approval, clarification or sensitive actions.
- Pause the graph, get human feedback and then continue.

```python
def human_input(state: AgentState):
    user_feedback = input(
        "Please review and provide input: "
    )
    return {"human_feedback": user_feedback}
graph.add_node("human", human_input)
```

Humans + AI = Better and safer decisions.

9. Practical Use Cases

- Multi step research agents
- Customer support workflows
- Content generation with review
- Tool using agents (search, calculator, apis)
- Multi agent systems
- Data analysis pipelines
- Autonomous AI assistants
- Complex decision making workflows

LangGraph is ideal for real world agent applications.

10. Key Takeaways

- LangGraph helps you build stateful and controllable agents.
- Understand nodes, edges and state.
- Use conditional edges for dynamic workflows.
- Supports loops and human in the loop.
- Works well with LangChain tools, memory and other components.
- Great for complex, real world applications.
- Start small, test well and then scale.

"From simple graphs to powerful agents LangGraph gives you control."

Plan Build Test Scale !

More Agents More Possibilities !

Multiple Agents Bigger Solutions Real Impact !

16. Multi Agent Systems + Workflows

@codedsoul_05

Plan Coordinate Execute Together !

1. What are Multi Agent Systems ?

- A system where multiple AI agents work together to solve complex tasks.
- Each agent has a specific role, goal and tools.
- Agents can communicate, collaborate and take actions independently.
- Useful for real world problems that need multiple skills like research, planning, execution and verification.
- LangChain and LangGraph make it easy to build multi agent systems.

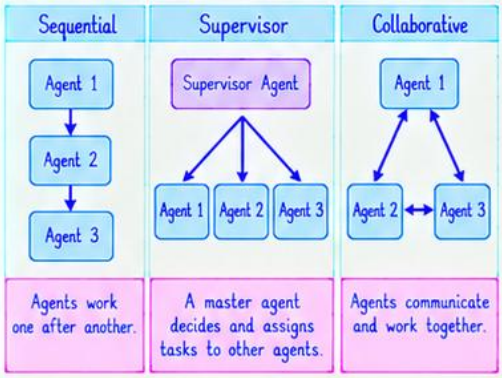
"Multiple agents working together achieve more than a single agent."

2. Why Use Multi Agent Systems ?

- Break complex problems into smaller tasks.
- Each agent can focus on a specific role.
- Improves accuracy and quality of results.
- Helps in parallel execution and faster completion.
- Useful for research, analysis, content creation, data processing, automation etc.
- Mimics real world team collaboration.

"Right agents for the right tasks lead to better results."

3. Common Architectures



4. Simple Multi Agent Example

```
from langchain_openai import ChatOpenAI
from langchain.agents import initialize_agent, Tool
from langchain.tools import tool

llm = ChatOpenAI(model="gpt-3.5-turbo")

@tool
def search(query: str) -> str:
    """Search the web."""
    return "Search results for: " + query

@tool
def summarize(text: str) -> str:
    """Summarize the given text."""
    return "Summary: " + text[:100]

tools = [search, summarize]

agent = initialize_agent(tools, llm, agent="zero-shot-react-description")

response = agent.invoke("Search and summarize latest AI news")
print(response)
```

Multiple agents can use different tools to complete a task.

5. Building with LangGraph

```
from langgraph_graph import StatOpeaph, START, END
from typing import TypedDict

class AgentState(TypedDict):
    task: str
    research: str
    summary: str
    final_answer: str

def research_agent(state: AgentState):
    # Agent 1 - Research
    return {"research": "researched data..."}

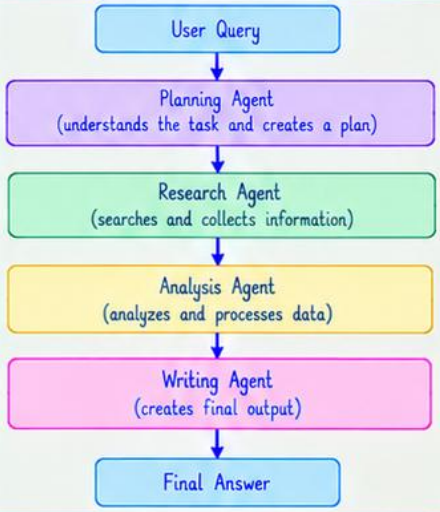
def summary_agent(state: AgentState):
    # Agent 2 - Summarize
    return {"summary": "summary of data..."}

def final_agent(state: AgentState):
    # Agent 3 - Final Answer
    return {"final_answer": "final response..."}

graph = StateGraph(AgentState)
graph.add_node("research", research_agent)
graph.add_node("summarize", summary_agent)
graph.add_node("final", final_agent)
graph.add_edge(START, "research")
graph.add_edge("research", "summarize")
graph.add_edge("summarize", "final")
graph.add_edge("final", END)
app = graph.compile()
```

LangGraph helps you define agent workflows using a graph.

6. Agent Workflow Example



Agents work in a structured workflow to complete complex tasks.

7. Real World Use Cases

- Research and report generation
- Customer support with multiple agents
- Data analysis and visualization
- Content creation (research, write, edit)
- Automated workflows for business tasks
- Multi step problem solving
- Autonomous agents for real world applications

Multi agent systems can handle complex tasks that are hard for a single agent.

8. Best Practices

- Define clear roles and goals for each agent
- Use structured workflows
- Handle errors and retries
- Limit the number of agents to avoid confusion
- Use memory for context sharing
- Monitor and log agent interactions
- Test with real use cases and iterate

Good planning and monitoring make multi agent systems reliable.

9. Key Takeaways

- Multi agent systems solve complex problems by breaking them into smaller tasks.
- LangGraph makes it easy to build stateful and controllable agent workflows.
- You can use sequential, supervisor or collaborative architectures.
- Each agent can have its own tools, memory and role.
- Useful for real world applications like research, automation, content creation and more.

"Agents together turn ideas into action."

More Agents More Possibilities !

Same Concepts Bigger Applications !

Trace Evaluate Improve Build Better AI !

17. LangSmith + Tracing + Evaluation

@codedsoul_05

Same Concepts Bigger Possibilities !

1. What is LangSmith ?

- LangSmith is a platform by LangChain for debugging, tracing, evaluating and monitoring LLM applications.
- It helps you understand what is happening inside your chain or agent.
- Provides observability for prompts, LLM calls, tools, agents and workflows.
- Helps you find issues, compare runs, improve prompts and measure performance.
- Useful for building reliable and production ready AI applications.

"LangSmith gives visibility into your LLM applications so you can debug, optimize and ship with confidence."

2. Why Tracing is Important ?

- Lets you see each step of your application (LLM calls, tool calls, prompts, outputs).
- Helps in debugging errors and unexpected behavior.
- Visualizes the flow of execution.
- Shows token usage, latency and cost.
- Useful for complex chains and agents where multiple steps are involved.
- Makes collaboration easier with shared runs and datasets.

"If you can't see what your AI is doing, you can't improve it."

3. Key Features of LangSmith

Tracing	Track and visualize each step of execution
Debugging	Inspect inputs, outputs, prompts and errors
Evaluation	Measure quality using predefined metrics
Datasets	Create and manage test datasets
Experimentation	Compare different prompts, models and chains
Monitoring	Track performance, cost and latency over time
Collaboration	Share runs and results with your team
Integrations	Works with LangChain, OpenAI and other providers

All in one platform to develop, test and improve your LLM applications.

4. Basic Setup

- Create a LangSmith account at https://smith.langchain.com
- Get your API key from settings.
- Set environment variables.

```
import os
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "your_api_key"
os.environ["LANGCHAIN_PROJECT"] = "my-project"
```

- Now all your LangChain runs will be automatically traced in LangSmith.

Just a few lines of code to get started.

5. Tracing an LLM Call (Example)

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

llm = ChatOpenAI(model="gpt-3.5-turbo")

response = llm.invoke([
    HumanMessage(content="Explain LangChain in simple words.")
])

print(response.content)
```

- This run will appear in your LangSmith project.
- You can see input, output, tokens, latency and more.

Run your code and see the trace in LangSmith.

6. Tracing Chains and Agents

```
from langchain.agents import initialize_agent, Tool
from langchain.tools import tool
from langchain_openai import ChatOpenAI

@tool
def search(query: str) -> str:
    return "Search results for: " + query

llm = ChatOpenAI(model="gpt-3.5-turbo")
agent = initialize_agent(
    tools=[search],
    llm=llm,
    agent="zero-shot-react-description",
    verbose=True
)
agent.invoke({"input":"Find latest news about AI."})
```

LangSmith traces every step including tool calls.

7. Evaluation in LangSmith

- Evaluation helps you measure the quality of your LLM outputs.
- You can use predefined evaluators or create custom ones.
- Common metrics:

Relevance	How relevant is the response
Correctness	Is the answer factually correct
Helpfulness	Is the response useful
Coherence	Is the response well structured
Groundedness	Does it use the given context
Custom Metrics	Define your own evaluation criteria

Evaluate. Compare. Improve.

8. Datasets and Experiments

- Create datasets with test examples (question, context, expected answer).
- Run experiments with different prompts, models or configurations.
- Compare results side by side.
- Helps you choose the best approach based on real data.

```
from langsmith import Client

client = Client()
client.create_dataset(
    dataset_name="my-dataset",
    examples=[
        {"input": "What is RAG?", "answer": "..."},
        {"input": "Explain agents.", "answer": "..."}
    ]
)
```

Test your ideas on real examples, not just vibes.

9. Best Practices

- Trace everything during development.
- Use meaningful run names and tags.
- Create datasets for important use cases.
- Evaluate regularly and compare results.
- Monitor token usage and cost.
- Collaborate with your team using shared projects.
- Set up alerts for errors or performance issues.
- Keep iterating based on evaluation results.

10. Key Takeaways

- LangSmith helps you build reliable and high quality LLM applications.
- Tracing gives visibility into every step.
- Evaluation helps you measure and improve performance.
- Datasets and experiments make it easy to compare ideas.
- Essential for real world and production deployments.

"Observe. Evaluate. Iterate. Build Better AI."

Better Traces Better Products !

Same Concepts Bigger Impact !

Safe Responsible Scalable AI Real Impact !

18. Security + Guardrails + Production

@codedsoul_05

Build Deploy Monitor Grow !

1. Why Security Matters ?

- LLM applications can deal with sensitive data (user information, business data, internal documents).
- They are vulnerable to misuse, prompt injection, data leaks and model abuse.
- Security ensures trust, compliance and safe usage in real world.
- Important for both users and businesses to adopt AI responsibly.
- A secure system protects data, users and your brand reputation.

"Great AI is not just powerful, it is also safe and trustworthy."

2. Common Risks and Threats

- Prompt Injection Manipulating the model to ignore instructions.
- Jailbreaking Bypassing safety restrictions.
- Data Leakage Accidental exposure of sensitive information.
- Insecure Tool Usage Harmful actions through connected tools.
- Adversarial Inputs Tricking the model with malicious prompts.
- Model Hallucination Incorrect or misleading outputs.
- Abuse and Misuse Spam, harmful content or illegal activities.

"Understand the risks so you can build better protection."

3. Guardrails and Mitigations

Guardrail	What it Does	Example / Tool
Input Validation	Checks and filters user input	Regex, LLM Guard
Content Moderation	Blocks harmful content	OpenAI Moderation, Llama Guard
Prompt Injection Detection	Detects malicious instructions	Guardrails AI, NeMo Guardrails
Output Filtering	Filters unsafe outputs	Custom filters, Moderation APIs
Tool Permissions	Restricts tool access and actions	Allowlist, Role based access
Data Privacy	Removes or masks sensitive data	PII redaction, DLP tools
Rate Limiting	Prevents abuse and overuse	API limits, quotas
Audit Logging	Tracks activity for security and debugging	LangSmith, Cloud logs

"Guardrails help you control what goes in, what happens inside and what comes out."

4. Input and Output Guardrails

```
# Simple input guardrail
def validate_input(user_input: str) -> str:
    banned_words = ["hack", "ignore", "bypass"]
    for word in banned_words:
        if word in user_input.lower():
            return "Your input contains restricted content."
    return user_input

# Output guardrail (using OpenAI moderation)
from openai import OpenAI
client = OpenAI()

def check_output(text: str) -> str:
    response = client.moderations.create(input=text)
    if response.results[0].flagged:
        return "Output blocked due to policy violation."
    return text
```

Validate inputs and filter outputs to keep your application safe.

5. Secure Tool Usage

- Give only required permissions to tools.
- Use allowlist instead of blocklist.
- Validate parameters before tool execution.
- Prevent tools from accessing sensitive systems directly.
- Log and monitor all tool calls.

```
# Example: Allow only specific tools
allowed_tools = ["web_search", "calculator"]

def run_tool(tool_name: str, args: dict):
    if tool_name not in allowed_tools:
        return "Tool not allowed."
    # execute tool safely
    return f"Running {tool_name} with {args}"
```

Least privilege access makes your agent safer and more reliable.

6. Deployment and Production

- Use environment variables for API keys and secrets.
- Deploy using secure cloud platforms (AWS, GCP, Azure).
- Enable HTTPS and encryption for data in transit.
- Use monitoring and logging (LangSmith, CloudWatch).
- Set up alerts for errors, unusual usage or policy violations.
- Plan for scalability, high availability and backups.
- Follow compliance requirements (GDPR, SOC2, etc.).

```
# Example: Load API key from environment
import os
from openai import OpenAI
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
```

A good deployment is secure, scalable and well monitored.

7. Monitoring and Observability

- Track requests, responses, token usage and latency.
- Use LangSmith for tracing and debugging.
- Monitor cost and performance.
- Set up alerts for anomalies.
- Review logs to detect misuse or unexpected behavior.
- Use dashboards for real time visibility.

What you monitor, you can improve.

8. Compliance and Responsible AI

- Follow data privacy laws (GDPR, DPDP, etc.).
- Use data anonymization and redaction.
- Be transparent about AI usage.
- Avoid biased or harmful outputs.
- Keep human in the loop for critical decisions.
- Document your system and policies.

Build AI that is ethical, fair and compliant.

9. Best Practices Checklist

- [ ] Validate and sanitize user inputs
- [ ] Use content moderation and output filters
- [ ] Restrict tool permissions with allowlist
- [ ] Store secrets in environment variables
- [ ] Enable logging and monitoring
- [ ] Set rate limits and usage quotas
- [ ] Plan for scalability and backups
- [ ] Follow compliance and privacy guidelines
- [ ] Test for security vulnerabilities
- [ ] Keep improving based on real world feedback

10. Key Takeaways

- Security, guardrails and production readiness are essential for real world AI applications.
- Protect your users, data and brand reputation.
- Use guardrails at every layer: input, model, tools and output.
- Monitor, log and iterate continuously.
- Follow best practices and compliance requirements.
- A safe and reliable AI application builds long term trust.
- Start small, test well and then scale.

"Safe AI today, a better tomorrow."

From Prototype to Production !

Same Concepts Bigger Possibilities !

From Idea to Deployment ! Real Project !

19. End to End LangChain RAG + Agent Project

@codedsoul_05

Learn Build Customize Deploy !

Build a Real World AI Application Step by Step

1. Project Overview

- Build a real world AI application using LangChain.
- Combine RAG (Retrieval Augmented Generation) with Agents.
- Use your own documents (PDF, Docs, URLs).
- Let the agent answer questions, summarize content, and perform actions using tools.
- Deploy the application with a simple UI and make it production ready.

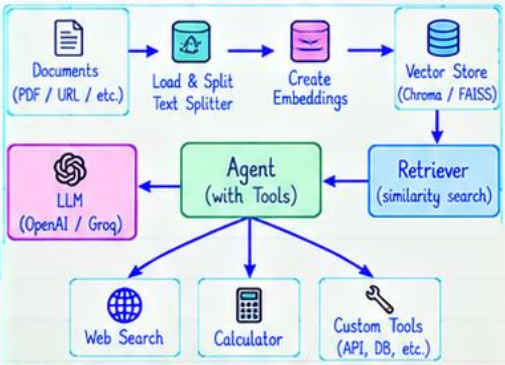
"A complete hands on project to go from data to intelligent actions."

2. Tech Stack

- Python
- LangChain, LangGraph (optional)
- OpenAI or any LLM (Groq, Anthropic, etc.)
- Vector Store (Chroma / FAISS / Pinecone)
- Document Loaders (PDF, URL, etc.)
- Streamlit (for UI)
- Tools (Web Search, Calculator, etc.)
- Deployment (Docker, Railway, Render, etc.)

Python · LangChain · OpenAI · Chroma · Streamlit · Docker · Render · GitHub

3. System Architecture



RAG gives knowledge. Agent gives action.

4. Load and Process Documents

```python
from langchain_community.document_loaders
import PyPDFLoader, WebBaseLoader
from langchain.text_splitter import
RecursiveCharacterTextSplitter

# Load PDF
loader = PyPDFLoader("data/notes.pdf")
docs = loader.load()

# Split documents
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
splits = splitter.split_documents(docs)
```

Load your data and split it into smaller chunks for better retrieval.

5. Create Embeddings and Vector Store

```python
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
# Create embeddings
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small"
)
# Store in Chroma
vectorstore = Chroma.from_documents(
    documents=splits,
    embedding=embeddings,
    persist_directory="chroma_db"
)
# Create retriever
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 5}
```

Convert your content into embeddings and store it in a vector database.

6. Build RAG Chain

```python
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.2)

qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    chain_type="stuff",
    return_source_documents=True
)

response = qa_chain.invoke({
    "query": "Explain the main points from the document."
})
print(response["result"])
```

RAG chain answers questions using your documents with source references.

7. Add Tools and Create Agent

```python
from langchain.agents import initialize_agent, Tool
from langchain.tools import DuckDuckGoSearchRun

search_tool = Tool(
    name="Web Search",
    func=DuckDuckGoSearchRun().run,
    description="Search the web for latest information"
)
tools = [search_tool]

agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent="zero-shot-react-description",
    verbose=True
)
agent.invoke({"input": "Search and summarize latest AI news."})
```

Give your agent tools so it can take actions like web search, calculation, or custom APIs.

8. Build a UI with Streamlit

```python
import streamlit as st

st.set_page_config(page_title="RAG Agent", page_icon="🤖")

st.title("📚 RAG + Agent Assistant")

query = st.text_input("Ask a question about your documents:")

if st.button("Get Answer"):
    response = agent.invoke({"input": query})
    st.write("### Answer")
    st.write(response["output"])
    if "source_documents" in response:
        st.write("### Sources")
        for doc in response["source_documents"]:
            st.write(doc.metadata.get("source", "N/A"))
```

Create a simple UI to interact with your RAG Agent and see results in real time.

9. Deploy and Make it Production Ready

- Use environment variables for API keys.
- Deploy on Render / Railway / AWS / GCP.
- Use Docker for consistent setup.
- Add authentication if required.
- Monitor with LangSmith.
- Handle errors and add logging.
- Scale with a better vector database like Pinecone.

```dockerfile
# Example Dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["streamlit", "run", "app.py", "--server.port=8501"]
```

Take your project from local development to a live, scalable application.

10. Key Takeaways

- You learned how to build a complete LangChain project from scratch.
- RAG helps your AI use private data.
- Agents can take actions using tools.
- This project can be extended with more tools, better UI and advanced workflows.

"Build real projects. That's how you become job ready."

Build Learn Deploy Grow !

Same Concepts Bigger Possibilities !

Revise
Practice
Be Interview Ready!

20. LangChain Cheat Sheet + 100 Interview Questions

@codedsoul_05

Same Concepts Bigger Opportunities!

LangChain 1.x | Build | Learn | Apply | Grow

1. LangChain Core Cheat Sheet

- Chat Models: ChatOpenAI, ChatAnthropic etc.
- Messages: SystemMessage, HumanMessage, AIMessage, ToolMessage
- Prompts: ChatPromptTemplate, FewShotPrompt
- Tools: @tool, Structured tools, Tool integration
- Retrievers: Vector store retrievers, RAG
- Embeddings: OpenAIEmbeddings, HuggingFace
- Vector Stores: Chroma, FAISS, Pinecone, etc.
- Output Parsers: StrOutputParser, Pydantic
- Streaming: .stream(), .astream()

"Small building blocks, powerful applications."

2. Modern Agent API

- create_agent is the current primary agent factory in LangChain 1.x
- Runs a model tool loop until no more tool calls are needed
- Supports custom tools, system prompts, memory and middleware
- Works with OpenAI, Anthropic and other model providers
- Handles tool calling, retries and conversations automatically

```
from langchain.agents import create_agent
agent = create_agent(
    model="gpt-4.1",
    tools=[search_tool],
    system_prompt="You are a helpful assistant."
)
response = agent.invoke({"input": "Your question"})
```

"Let the agent think, use tools, and solve."

3. LCEL (LangChain Expression Language)

- Chain components using | operator
- Everything is a Runnable
- Modular, composable and easy to debug
- Supports streaming, batching and async

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in simple words."
)
model = ChatOpenAI(model="gpt-4.1")
parser = StrOutputParser()

chain = prompt | model | parser
result = chain.invoke({"topic": "RAG"})
```

"Prompt | Model | Parser = Powerful Chain"

4. RAG Flow

Load Documents
↓
Split into Chunks
↓
Create Embeddings
↓
Store in Vector DB
↓
Retriever (smilarity search)
↓
LLM (with context)
↓
Answer

"Your data, AI's superpower."

5. Tool Calling Flow

1. Model decides to use a tool
↓
2. Tool is executed
↓
3. ToolMessage is returned to the model
↓
4. Model continues and generates final answer

"Tools turn AI from a talker into a doer."

6. LangGraph (Stateful Workflows)

- Build stateful, multi step agent systems
- Define state using TypedDict or Pydantic
- Create nodes, edges and conditional logic
- Supports loops, human in the loop
- Durable execution and production ready

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict

class AgentState(TypedDict):
    messages: list

graph = StateGraph(AgentState)
# add nodes and edges
graph.add_node("agent", agent_node)
graph.add_edge(START, "agent")
graph.add_edge("agent", END)
```

"From linear chats to real world workflows."

7. Middleware (Control Behavior)

- before_model: modify input before LLM call
- after_model: modify output after LLM call
- wrap_model_call: custom logic around model
- wrap_tool_call: control tool execution
- Use for retries, logging, guardrails, context editing and tool limits

```
from langchain.agents import create_agent

def before_model(state, runtime):
    state["messages"] = [add_system_msg(state)]
    return state
agent = create_agent(
    model="gpt-4.1",
    tools=tools,
    middleware=[before_model],
)
```

"Observe. Control. Make it safer."

8. Structured Output

- Use Pydantic, TypedDict or JSON schema
- ToolStrategy: model uses tools to format
- ProviderStrategy: native structured output
- Ensures reliable and consistent responses
- Useful for APIs, data extraction and agents

```
from pydantic import BaseModel
from langchain.agents import create_agent
class ProductInfo(BaseModel):
    name: str
    price: float
    in_stock: bool
agent = create_agent(
  model="gpt-4.1",
  response_format=ProductInfo  # or ToolStrategy
)
result = agent.invoke({"input": "Extract product info"})
```

"Structured output = Reliable AI."

9. Production Checklist

- ☐ Use environment variables for API keys
- ☐ Validate and sanitice inputs
- ☐ Add retries and error handling
- ☐ Set timeouts and rate limits
- ☐ Enable logging and tracing (LangSmith)
- ☐ Evaluate quality with test datasets
- ☐ Add guardrails and content moderation
- ☐ Monitor cost, latency and token usage
- ☐ Plan for scalablity and fallbacks
- ☐ Follow security and compliance (GDPR etc.)

"Build. Test. Monitor. Scale."

Interview Formula

Concept → Architecture → Code → Trade-offs → Production

"A complete answer shows depth, not just syntax."

Key Takeaways

- LangChain makes it easy to build LLM applications.
- Use the right tool for the right problem.
- RAG + Tools + Agents + Guardrails = Real value.
- LangGraph for complex, stateful workflows.
- Always evaluate, monitor and improve.
- Learn by building real projects.

"Learn
Build
Solve
Repeat

That's how you grow."

10. 100 Interview Questions (Category Wise)

Q1 - Q10 LangChain Basics

- Q1 What is LangChain?
- Q2 Why use LangChain?
- Q3 Key components?
- Q4 LangChain vs LlamaIndex?
- Q5 What is a Runnable?
- Q6 What is LangChain 1.x?
- Q7 Benefits and use cases?
- Q8 How does LangChain work?
- Q9 Real world applications?
- Q10 Limitations of LangChain?

Q11 - Q20 Models and Chat Models

- Q11 How to use ChatOpenAI?
- Q12 Temperature vs Top P?
- Q13 Different model providers?
- Q14 How to switch models?
- Q15 How to handle token limits?
- Q16 What is a message?
- Q17 System vs Human message?
- Q18 Multi modal models?
- Q19 Streaming with models?
- Q20 How to reduce cost?

Q21 - Q30 Prompts and Messages

- Q21 What is a prompt template?
- Q22 Few shot prompting?
- Q23 ChatPromptTemplate?
- Q24 Prompt vs system message?
- Q25 How to manage chat history?
- Q26 How to use placeholders?
- Q27 Best practices for prompts?
- Q28 Prompt injection risks?
- Q29 How to avoid hallucinations?
- Q30 Dynamic prompts?

Q31 - Q40 LCEL and Chains

- Q31 What is LCEL?
- Q32 Why use | operator?
- Q33 What is a Runnable?
- Q34 Chain vs LCEL?
- Q35 How to add parallel steps?
- Q36 How to stream in LCEL?
- Q37 How to pass multiple inputs?
- Q38 LCEL vs traditional chains?
- Q39 How to debug LCEL chains?
- Q40 Real world LCEL example?

Q41 - Q50 Output Parsers

- Q41 What is an output parser?
- Q42 StrOutputParser?
- Q43 Pydantic output parser?
- Q44 When to use structured output?
- Q45 ToolStrategy vs ProviderStrategy?
- Q46 How to handle parsing errors?
- Q47 JSON schema for outputs?
- Q48 Enforce output format?
- Q49 Function calling vs structured?
- Q50 Best practices?

Q51 - Q60 Embeddings, Vector Stores

- Q51 What are embeddings?
- Q52 Popular embedding models?
- Q53 What is a vector store?
- Q54 FAISS vs Chroma vs Pinecone?
- Q55 How to store and query?
- Q56 What is a retriever?
- Q57 Similarity search vs MMR?
- Q58 How to choose chunk size?
- Q59 Dense vs sparse embeddings?
- Q60 Metadata filtering?

Q61 - Q70 RAG

- Q61 What is RAG?
- Q62 Why use RAG?
- Q63 RAG architecture?
- Q64 How to improve RAG accuracy?
- Q65 Advanced retrieval techniques?
- Q66 Handling long documents?
- Q67 Evaluating RAG systems?
- Q68 Common RAG failures?
- Q69 How to add citations?
- Q70 Production considerations?

Q71 - Q80 Tools and Tool Calling

- Q71 What is a tool in LangChain?
- Q72 How to define a tool?
- Q73 How does tool calling work?
- Q74 What is ToolMessage?
- Q75 When to use tools?
- Q76 Handle tool errors?
- Q77 Restrict tool acces?
- Q78 Parallel tool calls?
- Q79 Custom tool examples?
- Q80 Best practices for tools?

Q81 - Q90 Agents and Agent Loops

- Q81 What is an agent?
- Q82 How does create_agent work?
- Q83 Agent loop explained?
- Q84 ReAct vs tool using agents?
- Q85 How to add memory?
- Q86 Multi agent systems?
- Q87 How to control agent behavior?
- Q88 How to evaluate agents?
- Q89 Common agent failure cases?
- Q90 Real world agent examples?

Q91 - Q100 LangGraph, Eval, Production

- Q91 What is LangGraph?
- Q92 LangChain vs LangGraph?
- Q93 How to build a workflow?
- Q94 Human in the loop?
- Q95 When to use LangGraph?
- Q96 What is LangSmith?
- Q97 How to evaluate LLM apps?
- Q98 How to secure tool calling?
- Q99 How to deploy a LangChain app?
- Q100 Best practices for production?

More Learning More Opportunities!

Same Ideas Bigger Impact!